Object Oriented Programming

Mihai Dascălu

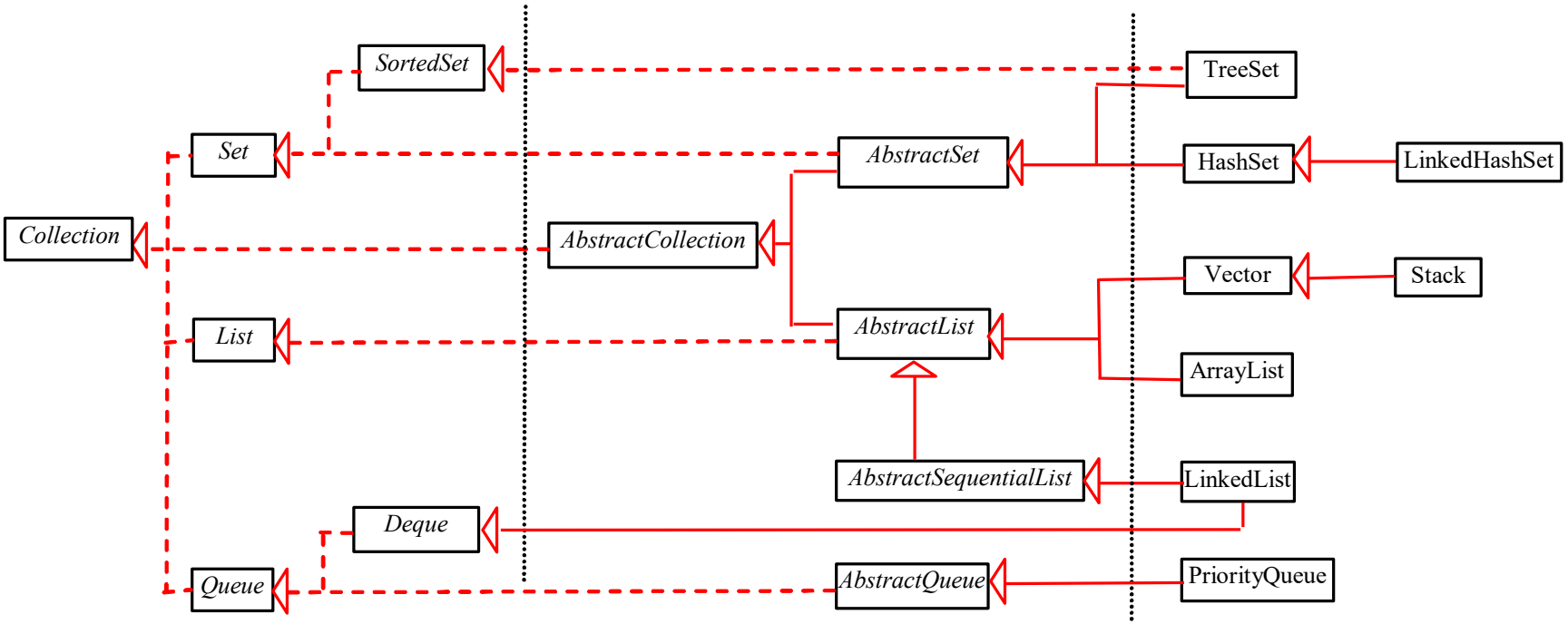
Collections

Java Collection Framework hierarchy (1)

- A **collection** = container object that represents a group of objects or elements.
- Three types of collections:
  - Sets
  - Lists
  - Maps

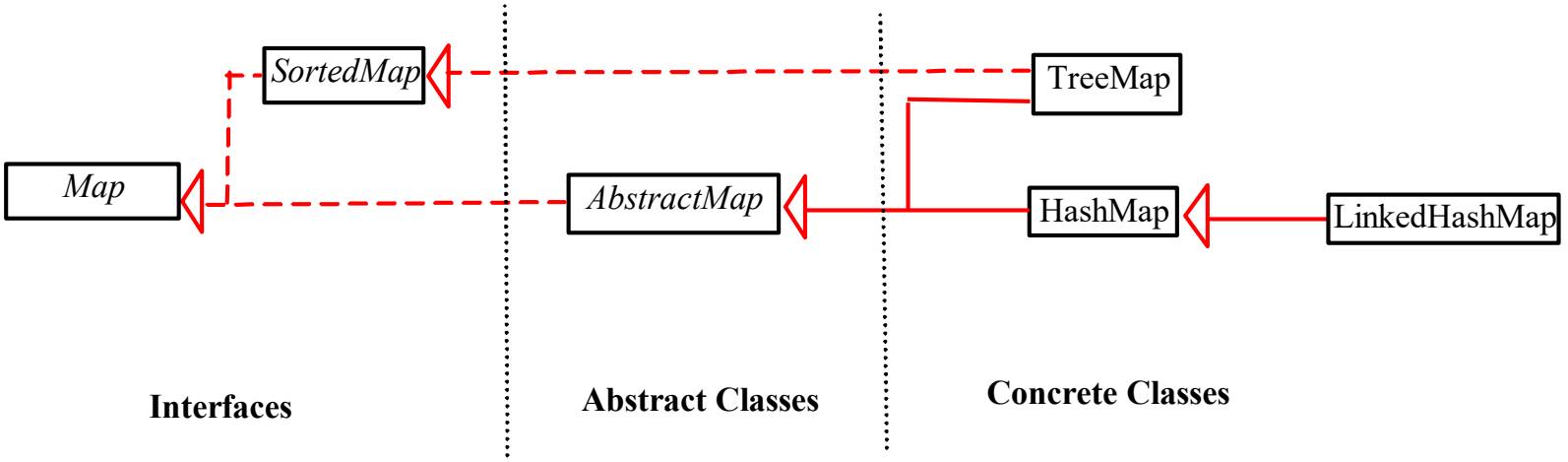
Java Collection Framework hierarchy (2)

Set and List are subinterfaces of Collection

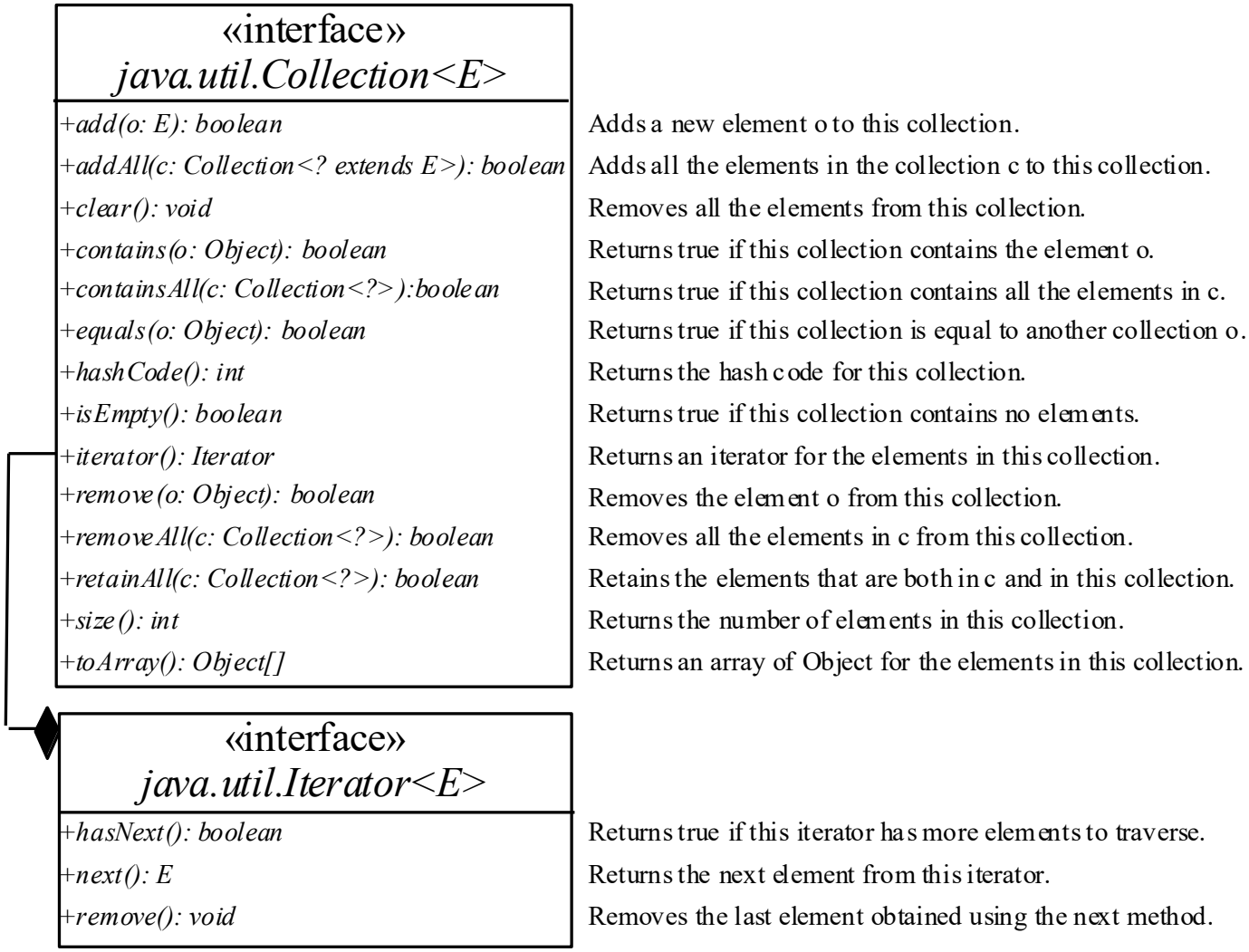


Java Collection Framework hierarchy (3)

- An instance of Map = a group of objects, each of which is associated with a key
- Get/put objects from a map using a key



The Collection Interface



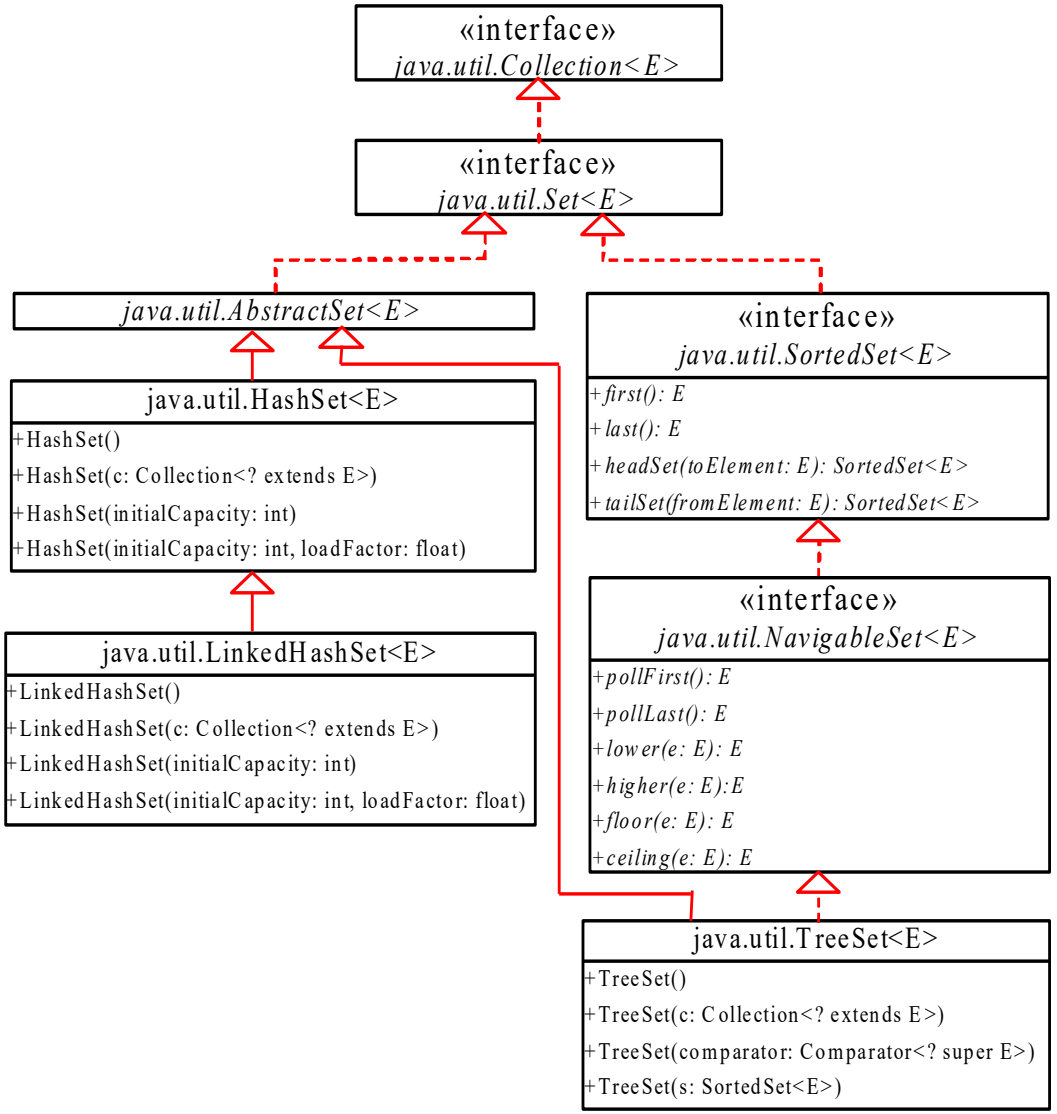
Adds a new element o to this collection.
Adds all the elements in the collection c to this collection.
Removes all the elements from this collection.
Returns true if this collection contains the element o.
Returns true if this collection contains all the elements in c.
Returns true if this collection is equal to another collection o.
Returns the hash code for this collection.
Returns true if this collection contains no elements.
Returns an iterator for the elements in this collection.
Removes the element o from this collection.
Removes all the elements in c from this collection.
Retains the elements that are both in c and in this collection.
Returns the number of elements in this collection.
Returns an array of Object for the elements in this collection.

Returns true if this iterator has more elements to traverse.
Returns the next element from this iterator.
Removes the last element obtained using the next method.

The Set Interface

- Extends the Collection interface
- Does not introduce new methods or constants, but it stipulates that an instance of Set contains no duplicate elements
- The concrete classes that implement Set must ensure that no duplicate elements can be added to the set
  - i.e., no two elements e1 and e2 can be in the set such that e1.equals(e2) is true

The Set Interface Hierarchy



The AbstractSet Class

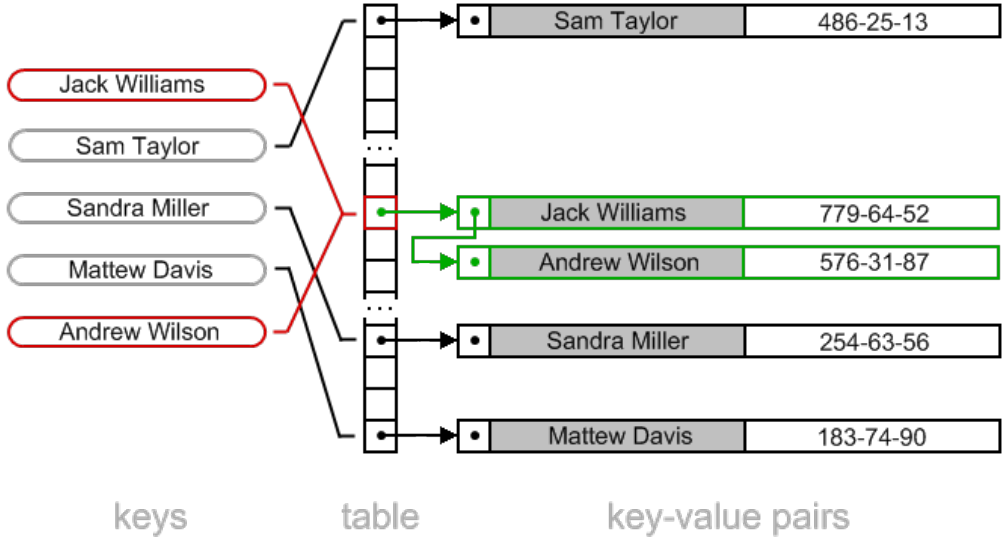
- The AbstractSet class is a convenience class that extends AbstractCollection and implements Set
- The AbstractSet class provides concrete implementations for the equals method and the hashCode method
- The hash code of a set is the sum of the hash code of all the elements in the set
- Since the size method and iterator method are not implemented in the AbstractSet class, AbstractSet is an abstract class

The HashSet Class

- The HashSet class is a concrete class that implements Set
- Used to store duplicate-free elements
- For efficiency, objects added to a hash set need to implement the hashCode method in a manner that properly disperses the hashcode

Hash collisions

- Hash value (.hashCode()) determines the bucket in which the object will be stored in
- Two or more key objects produce the same final hash value
  - Point to the same bucket location or array index
  - Two unequal objects !.equals() in Java can have the same hash code



Internal solution

- *Chaining*
  - Use linked list to store map entries which ended in the same bucket due to a collision
  - New entry is added at the head of the linked list there
  - Worst case scenario: O(n) instead of O(1)
- JDK8+ optimisation
  - Use a balanced tree instead of a linked list for storing collided entries
  - Performance boost from O(n) to O(log n)
  - TREEIFY_THRESHOLD = 8 / UNTREEIFY_THRESHOLD = 6 during resize()

TestHash

TIP: for-each loop

You can simplify the code using a JDK 1.5 enhanced for loop without using an iterator:

```
for (Object element: set)

  System.out.print(element.toString() + " ");
```

```
Iterator<Object> iterator = set.iterator();

while(iterator.hasNext())

  System.out.print(iterator.next().toString() + " ");
```

The SortedSet Interface and the TreeSet Class (1)

- SortedSet is a subinterface of Set, which guarantees that the elements in the set are sorted
- TreeSet is a concrete class that implements the SortedSet interface
- You can use an iterator to traverse the elements in the sorted order
- The elements can be sorted in two ways

The SortedSet Interface and the TreeSet Class (2)

- Use the Comparable interface

versus

- Specify a comparator for the elements in the set
  - if the class for the elements does not implement the Comparable interface

  or

  - you do not want to use the compareTo method in the class that implements the Comparable interface (order by comparator)

The Comparator Interface (1)

- Sometimes you want to insert elements of different types into a tree set
- The elements may not be instances of Comparable or are not comparable
- You can define a comparator to compare these elements
  - Create a class that implements the *java.util.Comparator* interface
  - The Comparator interface has two methods, compare and equals

The Comparator Interface (2)

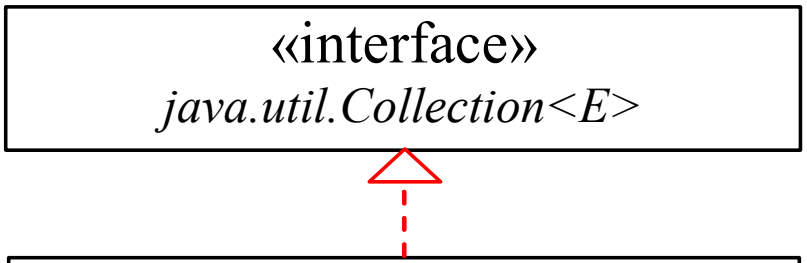
- public int compare(Object element1, Object element2)
  - Returns
    - Negative value if element1 is < than element2
    - Positive value if element1 is > than element2
    - Zero if they are equal
- public boolean equals(Object element)
  - Returns
    - True if the specified object is also a comparator and imposes the same ordering as this comparator

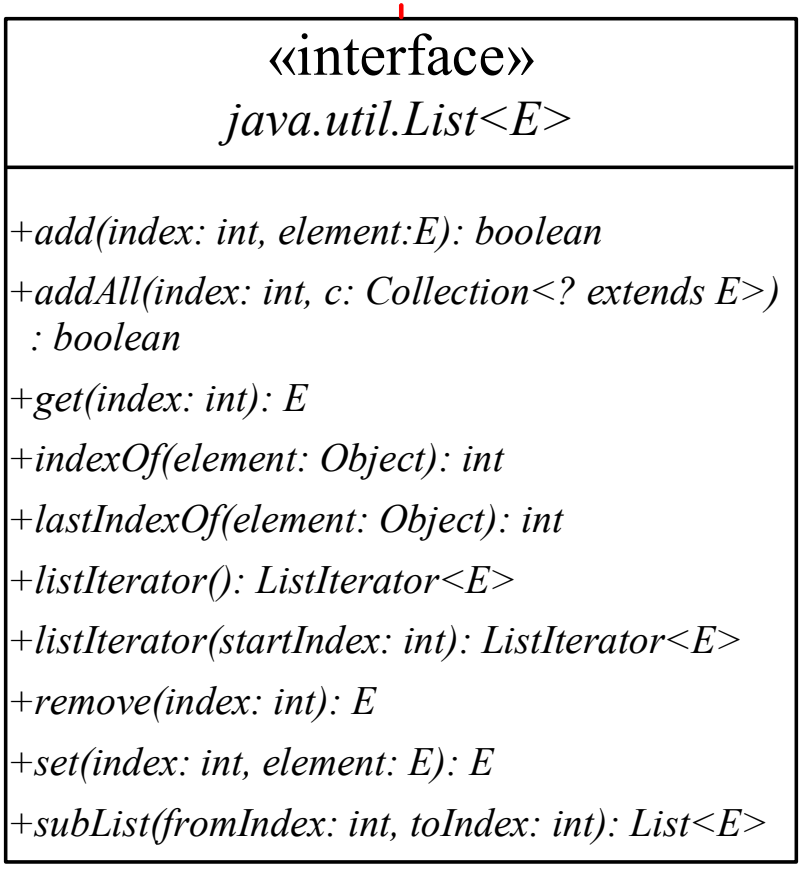
GeometricObjectComparator

The List Interface (1)

- A set stores non-duplicate elements
- A list is required to allow duplicate elements to be stored in a collection
- A list can not only store duplicate elements, but can also allow the user to specify where the element is stored
- The user can access the element by index

The List Interface (2)





Adds a new element at the specified index.

Adds all the elements in c to this list at the specified index.

Returns the element in this list at the specified index.

Returns the index of the first matching element.

Returns the index of the last matching element.

Returns the list iterator for the elements in this list.

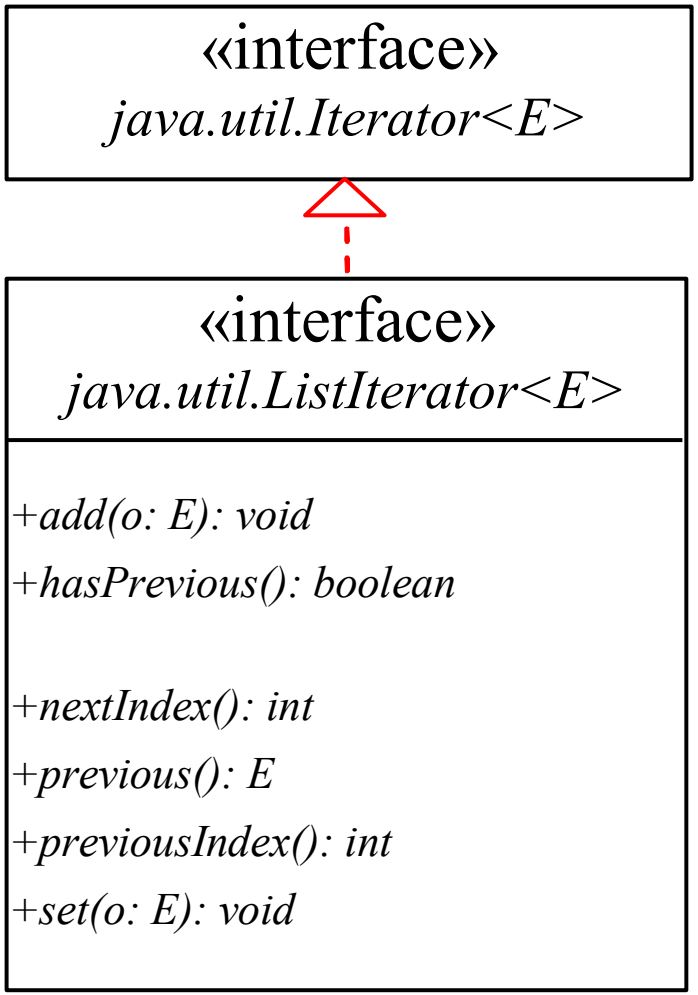
Returns the iterator for the elements from startIndex.

Removes the element at the specified index.

Sets the element at the specified index.

Returns a sublist from fromIndex to toIndex.

The List Iterator



Adds the specified object to the list.

Returns true if this list iterator has more elements when traversing backward.

Returns the index of the next element.

Returns the previous element in this list iterator.

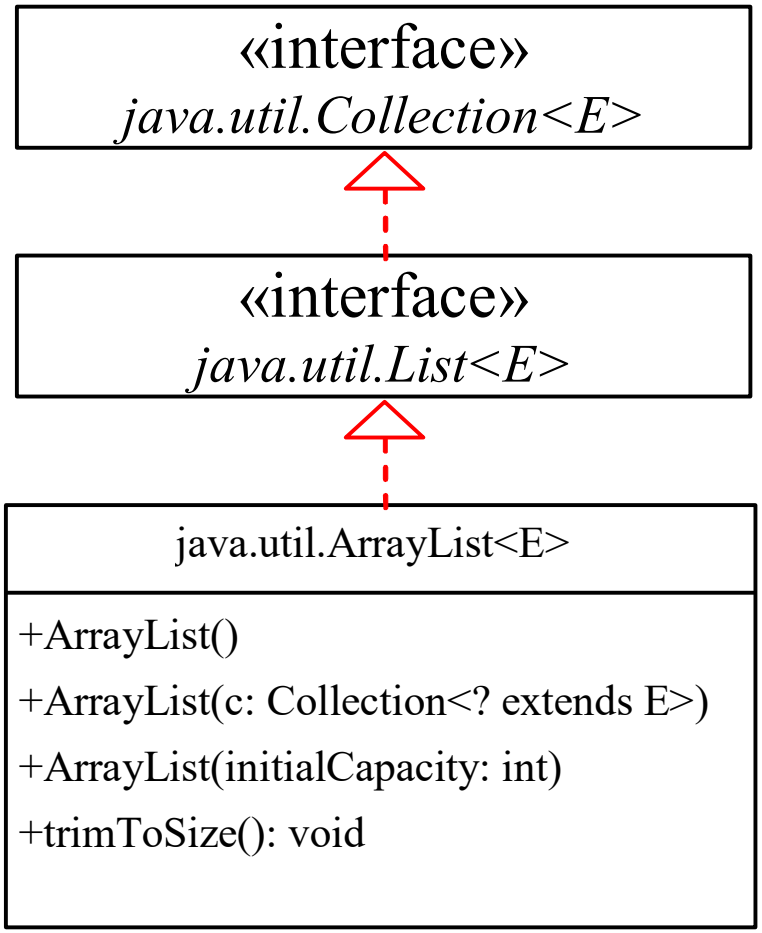
Returns the index of the previous element.

Replaces the last element returned by the previous or next method with the specified element.

ArrayList and LinkedList

- Concrete implementations of the List interface
- **ArrayList**: Random access through an index without inserting or removing elements from any place other than the end (fast get)
- **LinkedList**: Insertion or deletion of elements from any place in the list (fast remove)
- A list can grow or shrink dynamically
- An array [ ] is fixed once it is created
- If your application does not require insertion or deletion of elements, the most efficient data structure is the array

java.util.ArrayList



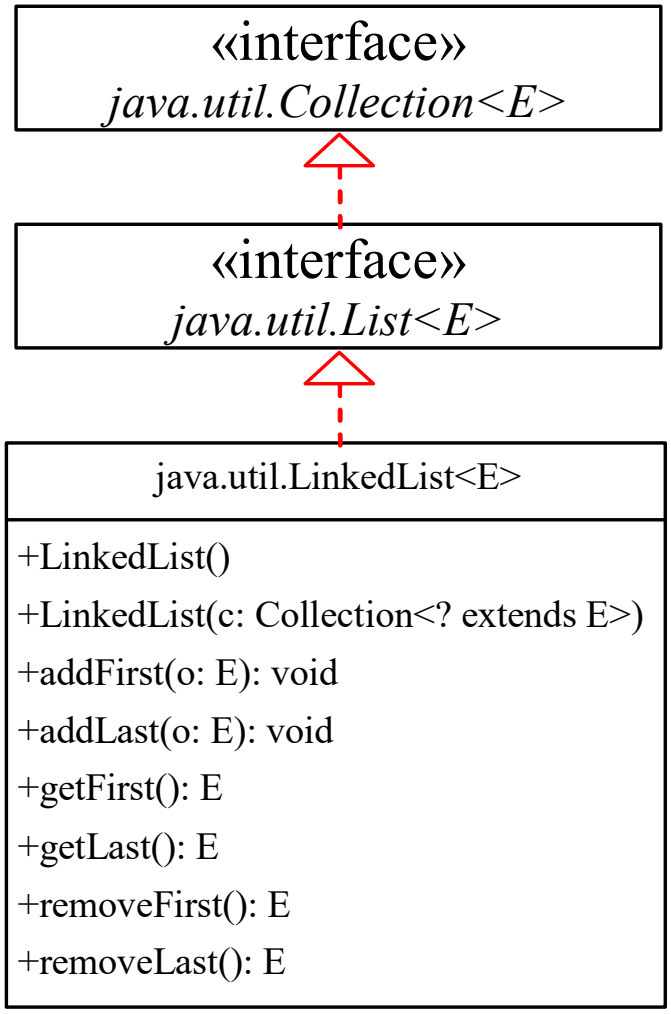
Creates an empty list with the default initial capacity.

Creates an array list from an existing collection.

Creates an empty list with the specified initial capacity.

Trims the capacity of this ArrayList instance to be the list's current size.

java.util.LinkedList

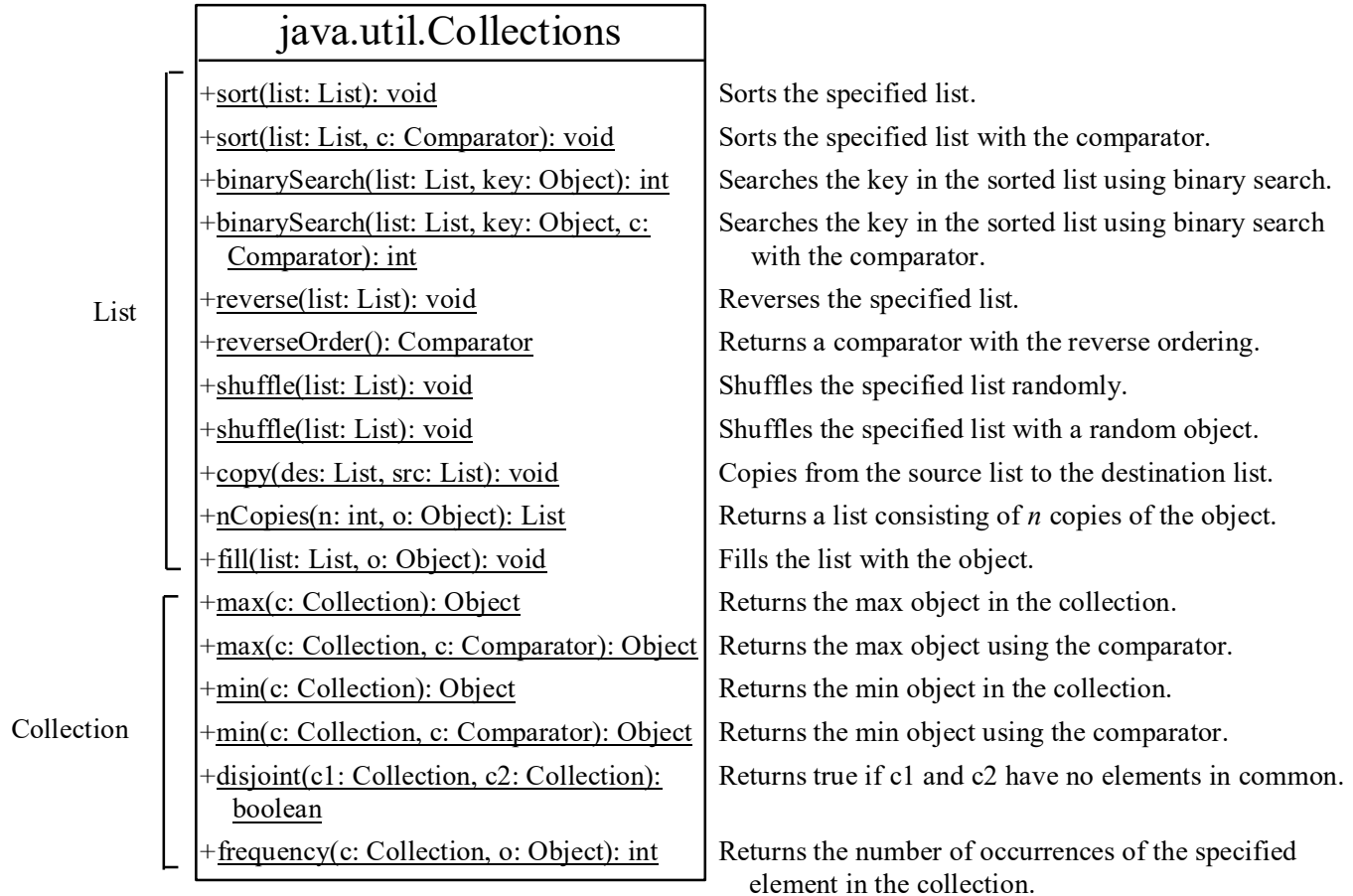


SetListPerformanceTest

Creates a default empty linked list.
Creates a linked list from an existing collection.
Adds the object to the head of this list.
Adds the object to the tail of this list.
Returns the first element from this list.
Returns the last element from this list.
Returns and removes the first element from this list.
Returns and removes the last element from this list.

The Collections Class

- Contains various static methods for operating on collections and maps, for creating synchronized collections, and for creating read-only collections

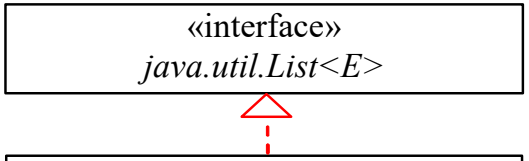


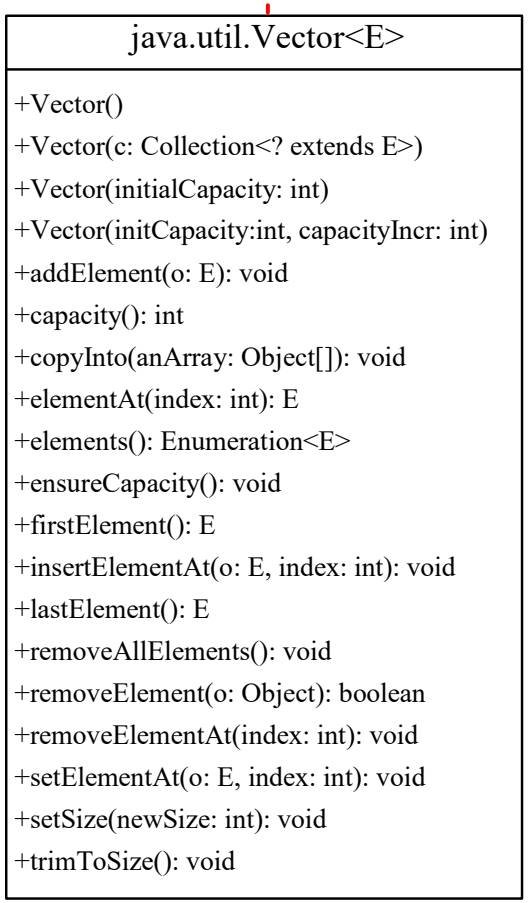
TestCollections

The Vector Class (1)

- Vector is the same as ArrayList, except that Vector contains the synchronized methods for accessing and modifying the vector
- None of the new collection data structures introduced so far are synchronized
- If synchronization is required, you can use the synchronized versions of the collection classes

The Vector Class (2)

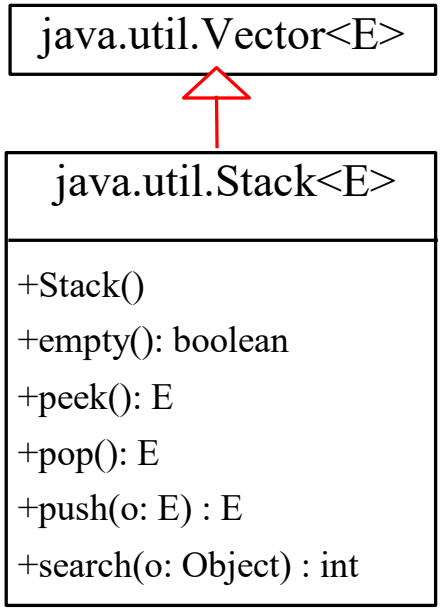




Creates a default empty vector with initial capacity 10.
Creates a vector from an existing collection.
Creates a vector with the specified initial capacity.
Creates a vector with the specified initial capacity and increment.
Appends the element to the end of this vector.
Returns the current capacity of this vector.
Copies the elements in this vector to the array.
Returns the object at the specified index.
Returns an enumeration of this vector.
Increases the capacity of this vector.
Returns the first element in this vector.
Inserts o to this vector at the specified index.
Returns the last element in this vector.
Removes all the elements in this vector.
Removes the first matching element in this vector.
Removes the element at the specified index.
Sets a new element at the specified index.
Sets a new size in this vector.
Trims the capacity of this vector to its size.

The Stack Class

- The Stack class represents a last-in-first-out stack of objects
- The elements are accessed only from the top of the stack
- You can retrieve, insert, or remove an element from the top of the stack



Creates an empty stack.

Returns true if this stack is empty.

Returns the top element in this stack.

Returns and removes the top element in this stack.

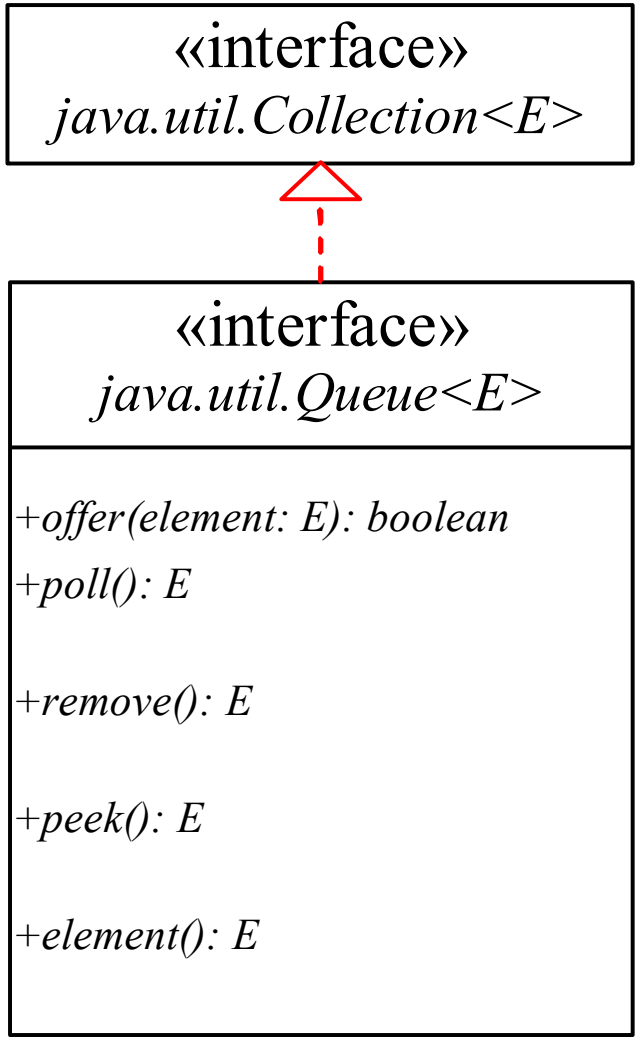
Adds a new element to the top of this stack.

Returns the position of the specified element in this stack.

Queues and Priority Queues

- First-in/first-out data structure
- Elements are appended to the end of the queue and are removed from the beginning of the queue
- In a priority queue, elements are assigned priorities
  - When accessing elements, the element with the highest priority is removed first

The Queue Interface



Inserts an element to the queue.

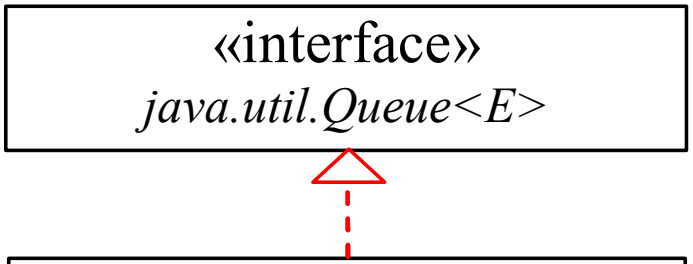
Retrieves and removes the head of this queue, or null if this queue is empty.

Retrieves and removes the head of this queue and throws an exception if this queue is empty.

Retrieves, but does not remove, the head of this queue, returning null if this queue is empty.

Retrieves, but does not remove, the head of this queue, throwing an exception if this queue is empty.

The PriorityQueue Class

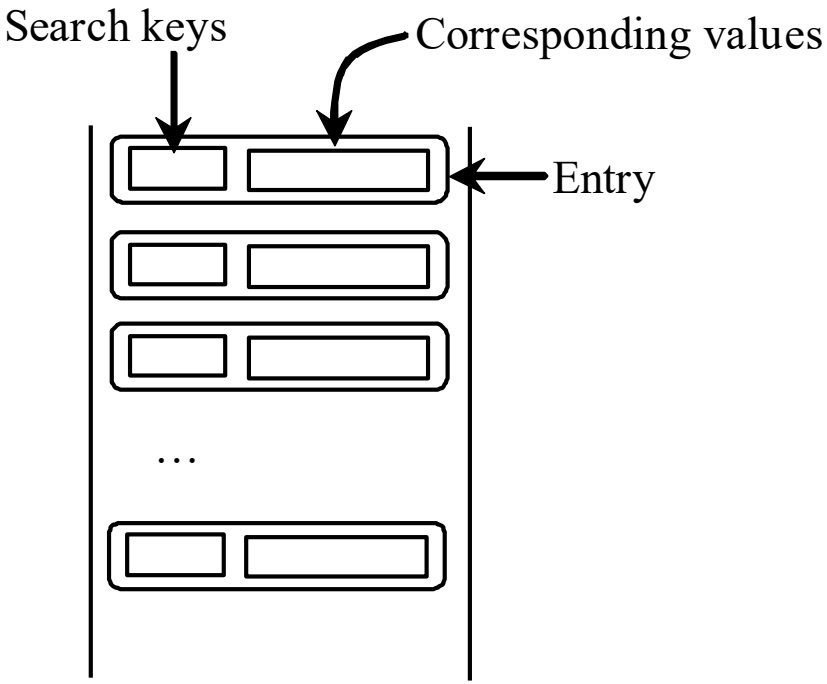


java.util.PriorityQueue<E>	
+PriorityQueue()	Creates a default priority queue with initial capacity 11.
+PriorityQueue(initialCapacity: int)	Creates a default priority queue with the specified initial capacity.
+PriorityQueue(c: Collection<? extends E>)	Creates a priority queue with the specified collection.
+PriorityQueue(initialCapacity: int, comparator: Comparator<? super E>)	Creates a priority queue with the specified initial capacity and the comparator.

PriorityQueueDemo

The Map Interface

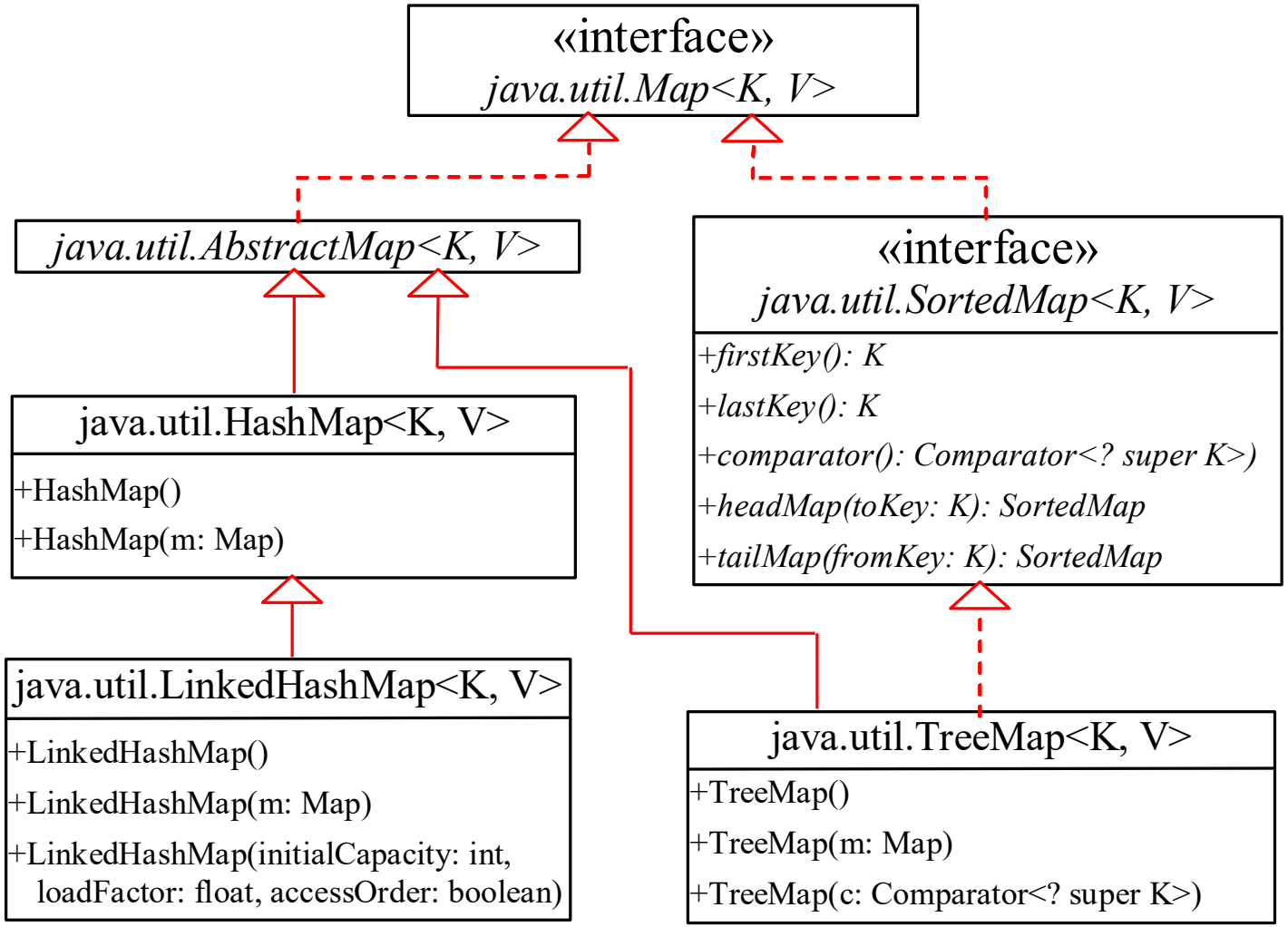
- Map keys to corresponding elements
- Keys can be perceived as indexes
  - In List, the indexes are integer
  - In Map, the keys can be any objects.



The Map Interface UML Diagram

java.util.Map<K, V>	
+clear(): void	Removes all mappings from this map.
+containsKey(key: Object): boolean	Returns true if this map contains a mapping for the specified key.
+containsValue(value: Object): boolean	Returns true if this map maps one or more keys to the specified value.
+entrySet(): Set	Returns a set consisting of the entries in this map.
+get(key: Object): V	Returns the value for the specified key in this map.
+isEmpty(): boolean	Returns true if this map contains no mappings.
+keySet(): Set<K>	Returns a set consisting of the keys in this map.
+put(key: K, value: V): V	Puts a mapping in this map.
+putAll(m: Map): void	Adds all the mappings from m to this map.
+remove(key: Object): V	Removes the mapping for the specified key.
+size(): int	Returns the number of mappings in this map.
+values(): Collection<V>	Returns a collection consisting of the values in this map.

Concrete Map Classes



HashMap and TreeMap

- The HashMap and TreeMap classes are two concrete implementations of the Map interface
- The HashMap class is efficient for locating a value, inserting a mapping, and deleting a mapping
- The TreeMap class, implementing SortedMap, is efficient for traversing the keys in a sorted order

```
for (Map.Entry<O1, O2> entry:map.entrySet())

  syso(entry.getKey()+" "+entry.getValue());
```

LinkedHashMap

- Extends HashMap with a linked list implementation that supports an ordering of the entries in the map
- Entries in a LinkedHashMap can be retrieved in the order:
  - they were inserted into the map (known as the insertion order),
  - they were last accessed, from least recently accessed to most recently (access order)
- The no-arg constructor constructs a LinkedHashMap with the insertion order
- To construct a LinkedHashMap with the access order, use: the LinkedHashMap(initialCapacity, loadFactor, true)

CountWordOccurrences

The Arrays Class

- Contains various static methods for sorting and searching arrays, for comparing arrays, for filling array elements and for converting an array to a list

Arrays	
+asList(a: Object[]): List	Returns a list from an array of objects
Overloaded binarySearch method for byte, char, short, int, long, float, double, and Object. +binarySearch(a: xType[], key: xType): int	Overloaded binary search method to search a key in the array of byte, char, short, int, long, float, double, and Object
Overloaded equals method for boolean, byte, char, short, int, long, float, double, and Object. +equals(a: xType[], a2: xType[]): boolean	Overloaded equals method that returns true if a is equal to a2 for a and a2 of the boolean, byte, char, short, int, long, float, and Object type
Overloaded fill method for boolean char, byte, short, int, long, float, double, and Object. +fill(a: xType[], val: xType): void +fill(a: xType[], fromIndex: int, toIndex: xType, val: xType): void	Overloaded fill method to fill in the specified value into the array of the boolean, byte, char, short, int, long, float, and Object type
Overloaded sort method for char, byte, short, int, long, float, double, and Object. +sort(a: xType[]): void +sort(a: xType[], fromIndex: int, toIndex: int): void	Overloaded sort method to sort the specified array of the char, byte, short, int, long, float, double, and Object type

TestArrays